

Experiment No. 6

Implementation and Comparative Analysis of Image Segmentation Techniques using Python and OpenCV

Aim

To implement and evaluate various image segmentation techniques for partitioning digital images into meaningful regions, thereby facilitating object localization, scene understanding, and subsequent computer vision tasks.

```
!pip install opencv-python scikit-learn -q
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

print("All libraries imported successfully!")
```

All libraries imported successfully!

```
from google.colab import files

uploaded = files.upload()

print("Image uploaded successfully!")
```

batman1.jpg
batman1.jpg(image/jpeg) - 116807 bytes, last modified: 9/15/2026 - 100% done
Saving batman1.jpg to batman1.jpg
Image uploaded successfully!

```
# Change the filename if your image has a different name
image = cv2.imread("batman1.jpg")

if image is None:
    print("Error: Image not found. Check the filename.")
else:
    print("Image loaded successfully!")
    print("Image dimensions:", image.shape)

    image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    plt.figure(figsize=(8, 6))
    plt.imshow(image_rgb)
    plt.title("Original Image")
    plt.axis("off")
    plt.show()
```

Image loaded successfully!
Image dimensions: (1200, 1200, 3)

Original Image



```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
plt.figure(figsize=(8, 6))  
plt.imshow(gray, cmap="gray")  
plt.title("Grayscale Image")  
plt.axis("off")  
plt.show()
```

Grayscale Image



```
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
plt.figure(figsize=(8, 6))  
plt.imshow(blurred, cmap="gray")  
plt.title("Gaussian Blurred Image")  
plt.axis("off")  
plt.show()
```

Gaussian Blurred Image



Observation

The image was converted into grayscale and Gaussian Blur was applied to reduce noise and small unwanted variations. This preprocessing helps improve the performance of segmentation techniques.

```
threshold_value = 127

_, global_threshold = cv2.threshold(
    blurred,
    threshold_value,
    255,
    cv2.THRESH_BINARY
)

plt.figure(figsize=(8, 6))
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")
plt.show()
```

Global Thresholding



Observation

Global Thresholding uses a single threshold value for the entire image. Pixels above the threshold are treated as foreground and pixels below the threshold are treated as background.

```
otsu_threshold_value, otsu_threshold = cv2.threshold(
    blurred,
    0,
    255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

print("Automatically selected Otsu threshold:", otsu_threshold_value)

plt.figure(figsize=(8, 6))
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")
plt.show()
```

Automatically selected Otsu threshold: 111.0

Otsu's Thresholding



✓ Observation

Otsu's Thresholding automatically calculates a suitable threshold value from the image histogram. It is useful when the image has a clear separation between foreground and background.

```
adaptive_threshold = cv2.adaptiveThreshold(
    blurred,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11,
    2
)

plt.figure(figsize=(8, 6))
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")
plt.show()
```

Adaptive Thresholding



Observation

Adaptive Thresholding calculates different threshold values for different regions of the image. It is useful when the image contains uneven lighting or illumination.

```
# Use Otsu thresholding as the binary image
binary = otsu_threshold.copy()

# Make sure objects are white and background is black
# If necessary, invert the image
white_pixels = np.sum(binary == 255)
black_pixels = np.sum(binary == 0)

if white_pixels > black_pixels:
    binary = cv2.bitwise_not(binary)

plt.figure(figsize=(8, 6))
plt.imshow(binary, cmap="gray")
plt.title("Binary Image for Watershed")
plt.axis("off")
plt.show()
```

Binary Image for Watershed



```

kernel = np.ones((3, 3), np.uint8)

# Remove small noise
opening = cv2.morphologyEx(
    binary,
    cv2.MORPH_OPEN,
    kernel,
    iterations=2
)

# Find sure background
sure_bg = cv2.dilate(
    opening,
    kernel,
    iterations=3
)

# Find sure foreground using distance transform
distance = cv2.distanceTransform(
    opening,
    cv2.DIST_L2,
    5
)

_, sure_fg = cv2.threshold(
    distance,
    0.5 * distance.max(),
    255,
    0
)

sure_fg = np.uint8(sure_fg)

# Unknown region
unknown = cv2.subtract(
    sure_bg,
    sure_fg
)

plt.figure(figsize=(12, 4))

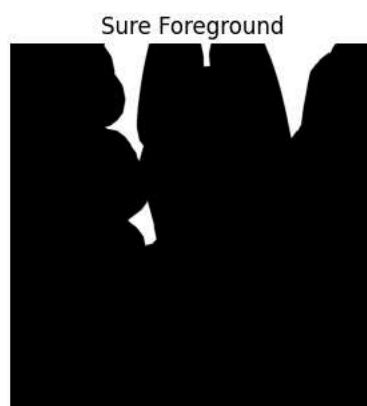
plt.subplot(1, 3, 1)
plt.imshow(opening, cmap="gray")
plt.title("Opening")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(sure_fg, cmap="gray")
plt.title("Sure Foreground")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(unknown, cmap="gray")
plt.title("Unknown Region")
plt.axis("off")

plt.show()

```



```

# Create markers
num_labels, markers = cv2.connectedComponents(sure_fg)

print("Number of detected foreground regions:", num_labels - 1)

# Add 1 so that background is not 0

```

```

markers = markers + 1

# Mark unknown region as 0
markers[unknown == 255] = 0

# Apply watershed
watershed_image = image.copy()

markers = cv2.watershed(
    watershed_image,
    markers
)

# Mark watershed boundaries in red
watershed_image[markers == -1] = [0, 0, 255]

watershed_rgb = cv2.cvtColor(
    watershed_image,
    cv2.COLOR_BGR2RGB
)

plt.figure(figsize=(10, 7))
plt.imshow(watershed_rgb)
plt.title("Watershed Segmentation")
plt.axis("off")
plt.show()

```

Number of detected foreground regions: 4

Watershed Segmentation



Observation

Watershed Segmentation treats the image like a topographic surface and separates different regions using boundaries. It is especially useful for separating touching or overlapping objects.

```

# Convert BGR image to RGB
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Reshape image into a list of pixels
pixel_data = image_rgb.reshape(
    (-1, 3)
)

# Convert to float
pixel_data = np.float32(pixel_data)

print("Total number of pixels:", len(pixel_data))

```

Total number of pixels: 1440000

```
# Number of clusters
```

```

k = 3

kmeans = KMeans(
    n_clusters=k,
    random_state=42,
    n_init=10
)

labels = kmeans.fit_predict(pixel_data)

centers = np.uint8(kmeans.cluster_centers_)

segmented_pixels = centers[labels]

segmented_image = segmented_pixels.reshape(
    image_rgb.shape
)

plt.figure(figsize=(8, 6))
plt.imshow(segmented_image)
plt.title("K-Means Color Segmentation")
plt.axis("off")
plt.show()

```

K-Means Color Segmentation



✓ Observation

K-Means Clustering groups pixels having similar color values into different clusters. Each pixel is assigned to the cluster with the closest color. This technique is useful for color-based image segmentation.

```

plt.figure(figsize=(16, 10))

plt.subplot(2, 3, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(global_threshold, cmap="gray")
plt.title("Global Thresholding")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(otsu_threshold, cmap="gray")
plt.title("Otsu's Thresholding")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(adaptive_threshold, cmap="gray")
plt.title("Adaptive Thresholding")
plt.axis("off")

plt.subplot(2, 3, 5)

```

```

plt.imshow(watershed_rgb)
plt.title("Watershed Segmentation")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(segmented_image)
plt.title("K-Means Clustering")
plt.axis("off")

plt.tight_layout()
plt.show()

```



```

techniques = [
    "Global Thresholding",
    "Otsu's Thresholding",
    "Adaptive Thresholding",
    "Watershed",
    "K-Means"
]

advantages = [
    "Simple and fast",
    "Automatically selects threshold",
    "Works with uneven illumination",
    "Separates touching objects",
    "Works well with color information"
]

limitations = [

```

```

    "Sensitive to threshold value",
    "Works best with clear foreground/background",
    "More computationally expensive",
    "Can over-segment",
    "Number of clusters must be selected"
]

for i in range(len(techniques)):
    print(f"{i+1}. {techniques[i]}")
    print("    Advantage :", advantages[i])
    print("    Limitation:", limitations[i])
    print()

```

1. Global Thresholding
 Advantage : Simple and fast
 Limitation: Sensitive to threshold value
2. Otsu's Thresholding
 Advantage : Automatically selects threshold
 Limitation: Works best with clear foreground/background
3. Adaptive Thresholding
 Advantage : Works with uneven illumination
 Limitation: More computationally expensive
4. Watershed
 Advantage : Separates touching objects
 Limitation: Can over-segment
5. K-Means
 Advantage : Works well with color information
 Limitation: Number of clusters must be selected

Comparison of Segmentation Techniques

Technique	Main Principle	Advantage	Limitation
Global Thresholding	Single threshold value	Simple and fast	Sensitive to threshold
Otsu's Thresholding	Automatically selects threshold	No manual threshold required	Works best with clear intensity separation
Adaptive Thresholding	Local threshold values	Handles uneven illumination	More computationally expensive
Watershed	Region/boundary based segmentation	Separates touching objects	Can cause over-segmentation
K-Means	Groups similar pixels	Good for color segmentation	Number of clusters must be selected

Observations

1. Global Thresholding separates the foreground and background using a fixed threshold value.
2. Otsu's Thresholding automatically determines a suitable threshold from the image histogram.
3. Adaptive Thresholding performs better when illumination is not uniform.
4. Watershed Segmentation is useful for separating touching or overlapping objects.
5. K-Means Clustering groups pixels according to their color similarity.
6. Threshold-based methods are simple and computationally efficient for suitable images.
7. Watershed is more useful when individual objects need to be separated.
8. K-Means is useful when color information is important.
9. The best segmentation method depends on the characteristics of the image.

Applications of Image Segmentation

Image segmentation is widely used in:

1. Medical image analysis and tumor detection
2. Satellite image analysis
3. Autonomous vehicles
4. Industrial quality inspection
5. Object detection and recognition
6. Face and biometric recognition
7. Agricultural image analysis
8. Traffic sign and road detection

Conclusion

In this experiment, different image segmentation techniques were implemented using Python, OpenCV, NumPy, Matplotlib, and Scikit-learn. Global Thresholding, Otsu's Thresholding, Adaptive Thresholding, Watershed Segmentation, and K-Means Clustering were applied to the selected image.

Each technique has different advantages and limitations. Thresholding methods are simple and effective for separating foreground and background, Watershed is useful for separating touching objects, and K-Means is useful for color-based segmentation. Therefore, the selection of a segmentation technique depends on the image characteristics and the required computer vision application.

Questions and Answers

1. What is image segmentation? Why is it considered a fundamental step in computer vision?

Image segmentation is the process of dividing an image into meaningful regions or objects. It helps separate the required object from the background. It is important because it makes further tasks such as object detection, recognition, tracking, and image analysis easier.

2. Differentiate between Image Segmentation and Image Classification.

Image Segmentation divides an image into different regions or pixels belonging to different objects.

Image Classification assigns a label to the complete image or an object.

For example, segmentation can identify the exact pixels belonging to a car, while classification can simply say that the image contains a car.

3. Explain Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Global Thresholding uses one fixed threshold value for the complete image.

Otsu's Thresholding automatically calculates an appropriate threshold value using the image histogram.

Adaptive Thresholding calculates different threshold values for different local regions of the image. It is useful for images with uneven illumination.

4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

Watershed is a segmentation technique that treats an image like a topographic surface. It identifies boundaries between different regions.

It is useful for separating touching or overlapping objects because it can find boundaries between individual objects.

5. How is K-Means Clustering applied to image segmentation?

In K-Means image segmentation, pixels are treated as data points. Pixels with similar color values are grouped into clusters. The value of K represents the number of clusters. Each pixel is assigned to the nearest cluster center, producing a segmented image.

6. Compare threshold-based segmentation and clustering-based segmentation techniques.

Threshold-based segmentation separates pixels using intensity or color threshold values. It is simple and fast.

Clustering-based segmentation groups pixels according to their similarity. It can work with multiple colors or regions but generally requires more computation.

7. What challenges are encountered while segmenting images with complex backgrounds or varying illumination?

Some common challenges are:

- Uneven lighting
- Shadows
- Noise
- Similar colors between object and background
- Complex backgrounds
- Object overlap
- Low contrast

These conditions can make it difficult to correctly separate the required object.

8. Mention five real-world applications where image segmentation plays a critical role.